

UNIVERSITY OF COMPUTER STUDIES, YANGON

2025-2026 ACADEMIC YEAR



Project Report On

LOG-BASED WEB ATTACK CLASSIFICATION

Semester IX

DATA AND KNOWLEDGE MINING (IS-212) PROJECT

Submitted By

YKPT-22443 Si Thu Aung

September, 2026

B.C.Sc.(CSF)

Table of Contents

Table of Contents	2
Acknowledgement	4
Chapter 1	5
Introduction	5
1.1 Project Background.....	5
1.2 Objectives of the Project.....	5
1.3 Tools and Environment.....	6
Chapter 2	7
Dataset Description and Exploratory Data Analysis	7
2.1 Dataset Overview	7
2.2 Log Parsing and Data Dictionary.....	7
2.3 Data Quality Check.....	8
2.4 Ground-Truth Labels (Target Variable).....	8
2.5 Exploratory Data Analysis.....	9
Chapter 3	12
Data Preprocessing and Descriptive Mining.....	12
3.1 Labelling Pipeline and Problems Solved	12
3.2 Feature Engineering.....	13
3.3 Descriptive Mining - K-Means Clustering	13
Table 3.2: The inertia and silhouette score for k ranging from 2 to 10	14
3.4 Best-K Selection via Combined Elbow and Silhouette	15
Chapter 4.....	17
Predictive Mining and Evaluation	17
4.1 Models and Pipeline Design	17
4.2 Problems Solved During Mining	17
4.3 Test Set Results.....	18
4.4 Cross-Validation Results	20
4.5 Multi-class View - Per-Attack-Type Precision and Recall.....	20
Table 4.4: The classification report for each class in the multi-class Random Forest.....	21
4.6 The Implemented System	21

Chapter 5	23
Conclusion	23
References	25

Acknowledgement

I would like to thank my instructor for helping us with the IS-212 lectures and for answering my questions in the lab sessions. It was the parts of the lectures covering classification, clustering, class imbalance, and model evaluation that formed the basis of this project. Moreover, the feedback I got when planning the project enabled me to steer clear of a number of possible mistakes.

I also need to give my thanks to the authors of the Web Server Log Dataset, available on Zenodo, as it was only because they provided an open and realistically generated collection of web-server logs together with a documented ground truth that students such as myself were able to carry out a project on web-attack detection.

I would like to thank my parents for their support during the semester and also to my classmates for testing the early versions of my notebook and for providing me with honest feedback every time a chart was confusing or a paragraph wasn't clear.

Chapter 1

Introduction

This report is part of the IS-212 Data and Knowledge Mining course and represents a personal project in which both descriptive and predictive data mining techniques are applied to a real network-security data set; the complete implementation is provided in a Jupyter notebook that can be executed end to end on Google Colab. All the material described in the chapters that follow, together with all the figures, was produced by the notebook using the original log files.

1.1 Project Background

For every request that the web server receives, there is an entry in its access logs. When an attacker is looking for vulnerabilities on a website, the indications of this kind of activity are usually obvious in the log entries—for example, the presence of SQL injection strings, cross-site scripting payloads, attempts to access directories by using paths such as `../`, and requests for known web shells. If it is possible to automatically detect such requests, the logs then act as a kind of intrusion-detection signal instead of merely functioning as a passive record. The project consists of developing a classifier which looks at the original Apache access log entries and determines whether each request is an attack or normal traffic and also identifies the type of attack it is.

The dataset that I have used is Organisation X's Apache web access log since it contains not only normal browsing behaviour but also the attacks which have been injected. It is available from the Web Server Log Dataset on Zenodo (record 20001206) and comprises realistic Apache access, error, auth and syslog files which were generated for a number of simulated organisations.

1.2 Objectives of the Project

The objectives of this project are the following:

- The aim is to develop a fully functional and reproducible data-mining pipeline that starts with the original (uncompressed and gzipped) Apache log files and ends with the evaluated models, with the intention of using them on Google Colab.
- It is necessary to create a clear table that displays the results of the analysis of the unstructured log lines, each request to be classified as either an attack or as benign on the basis of the classification rules contained in the dataset.

- Descriptive mining was applied—more precisely using K-Means clustering (with the elbow method and the silhouette method)—in order to find out whether attacks could be distinguished from normal traffic in the absence of labels.
- An identical evaluation procedure should be applied in the case of the three classifiers—Logistic Regression as the baseline, Random Forest, and a linear Support Vector Machine.
- When the target variable is imbalanced, it is necessary to use balanced accuracy, F1, MCC and AUC rather than depending solely on accuracy.
- In order to give the security analyst a multi-class view, it is necessary for the precision and recall values for each attack type to be made available.

1.3 Tools and Environment

The project makes use of a single Jupyter notebook on Google Colab, running on a standard CPU. The main libraries employed are pandas and NumPy for handling the data, scikit-learn for the models, the evaluation metrics and for K-Means, PyYAML for reading the ground-truth rules, and matplotlib and seaborn for the charts. The labelled dataset is written back out to a CSV file in order that the modelling stages may be reused without having to re-run the labeller, and the final model is saved using joblib. A fixed random seed (`random_state = 42`) is applied throughout the process to ensure that the same results are obtained each time.

Chapter Summary

This chapter has introduced the project by motivating the use of Apache access logs as a forensic artefact for web-attack detection, stating the six objectives that govern the work (an end-to-end reproducible pipeline, descriptive K-Means clustering, three supervised classifiers, honest evaluation under class imbalance, and a multi-class per-attack-type view), and listing the tools and environment (Google Colab, pandas, NumPy, scikit-learn, matplotlib and seaborn). The next chapter describes the dataset itself and presents the exploratory analysis that motivates the labelling and feature-engineering decisions.

Chapter 2

Dataset Description and Exploratory Data Analysis

2.1 Dataset Overview

The Apache access and error logs, the Linux auth and syslog files, and also a ground-truth file are included in the collection of Organization X and are stored in its Zenodo archive; the project has decided to concentrate on the Apache web-access.log since attacks over the web are found in the HTTP request log and the fact that this log is in the standard combined log format.

IP - - [timestamp] "METHOD URL PROTOCOL" status size "referer" "user-agent"

The access log includes 5,000 lines of requests; meanwhile, the auth.log and syslog files follow the standard Linux format (MMM DD HH:MM:SS host process[pid]: message), and, as is the case with Organization X, they are primarily made up of normal CRON messages and are therefore not used as a source of data. Since some of the files in the archive are gzipped but others use Windows CRLF line endings, a file reader has been created that can deal with both situations and so the pipeline remains accurate regardless of which rotated logs are being processed.

2.2 Log Parsing and Data Dictionary

I wrote a parser around a regular expression that captures the client address, timestamp, request, status, size, referer and user agent from each line. The request field is matched with a non-greedy pattern (.*) so that attack payloads which contain literal quote characters (common in XSS, SQL injection and RCE) are still parsed correctly. The parser achieved 100% coverage on the 5,000 lines, with zero unparseable lines. Each line is split into the nine columns listed in Table 2.1.

Table 2.1: Columns of the parsed dataset

Column (raw)	Type	Description
ip	str	Client IP address of the request
ts	str	Timestamp in Apache format DD/Mon/YYYY:HH:MM:SS +ZZZZ
method	str	HTTP method (GET, POST, HEAD, ...)
url	str	Requested URL (path and optional query string)
status	int	HTTP status code (200, 302, 304, 400, 404, 500, ...)
size	str	Response size in bytes (read as text, converted later)
referer	str	Referer header (or "-")
ua	str	User-Agent header
raw	str	Full original log line

2.3 Data Quality Check

I looked at the table to check for missing values, duplicate entries and any issues concerning data types. None of the columns contain any missing values, there are no identical rows, and the only case of a repeated request is when the same request is made twice at different timestamps. The status column is already in numeric form and comprises all the values from 200 to 500, having a mean of approximately 290 and a median of 302. Although the size column is initially read as text since some log formats use a dash to indicate empty responses, in this file every entry is a valid byte count, so it is converted into an integer as part of the feature engineering process.

2.4 Ground-Truth Labels (Target Variable)

The dataset includes a file called organization-x.yaml, and this is a ground-truth file which contains 39 labelling rules; each of these rules associates a set of substrings (which may be found in the method, the URL, the user agent or the status code) with an attack type together with a sensitivity level; the target variable is derived by applying the rules to the parsed table, since each request is

classified as either being benign or as one of the attack types, for example SQL injection, cross-site scripting, path traversal, remote code execution and directory scanning.

3,940 of the requests (78.8 per cent) are considered to be benign and 1,060 (21.2 per cent) are categorised as attacks. The most frequent types of attacks are `rce_read_file` (in the case of file disclosure via `/etc/passwd`), the different directory-scanning types (`dir_scan`, `dir_scan_go`, `dir_scan_python`), `remote_code` and `sql_injection_attempt`. The complete distribution is given in Table 2.2. Since the attack class is a minor category, the evaluation should not be based solely on simple accuracy, a point to which I refer in Chapter 4.

Table 2.2: Label distribution after applying the ground-truth rules

Label	Count	Share
benign	3,940	78.80%
<code>rce_read_file</code>	183	3.66%
<code>dir_scan_go</code>	171	3.42%
<code>dir_scan</code>	170	3.40%
<code>remote_code</code>	154	3.08%
<code>dir_scan_python</code>	151	3.02%
<code>sql_injection_attempt</code>	129	2.58%
<code>rce_java</code>	64	1.28%
<code>cross_site_scripting</code>	30	0.60%
<code>rce_shell</code>	8	0.16%
Total attacks	1,060	21.20%

2.5 Exploratory Data Analysis

As shown in Figure 2.1, GET requests make up the majority of the traffic, there being only a small number of POST and HEAD requests, which is the typical scenario for a web server. Figure 2.2 compares the status codes for benign and attack requests and shows that the benign traffic is primarily made up of 200 (OK), 302 (redirect) and 304 (not modified), whereas the attacks are mainly associated with 404 (not found, due to directory scanning) and 400 (bad request). Figure 2.3 gives the distribution of the various types of attack and indicates that file disclosure and directory scanning are the most common kinds of attack.

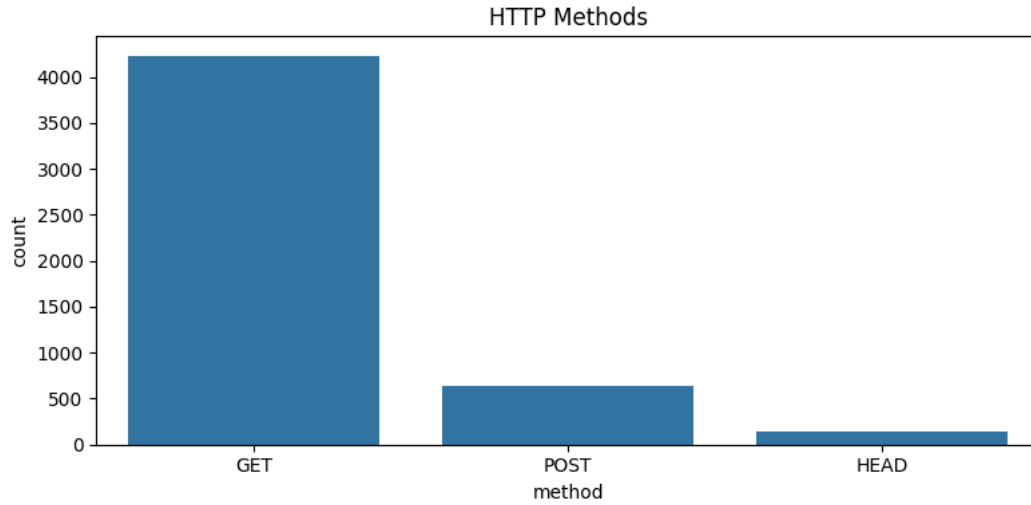


Figure 2.1: HTTP method distribution

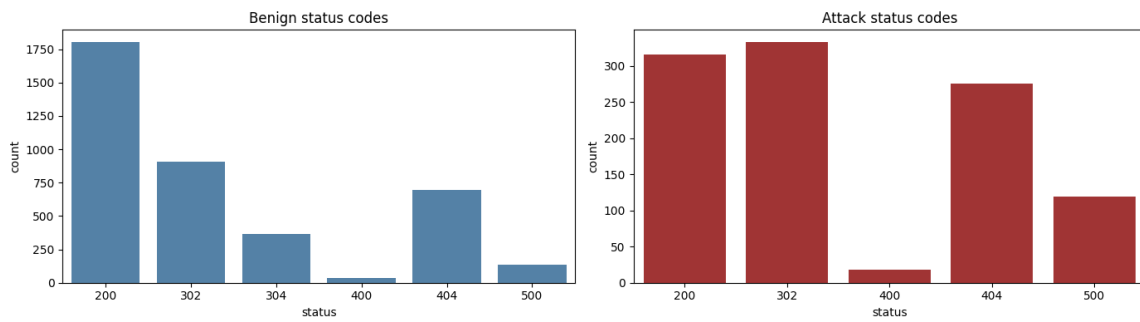


Figure 2.2: Status codes for benign (left) and attack (right) requests

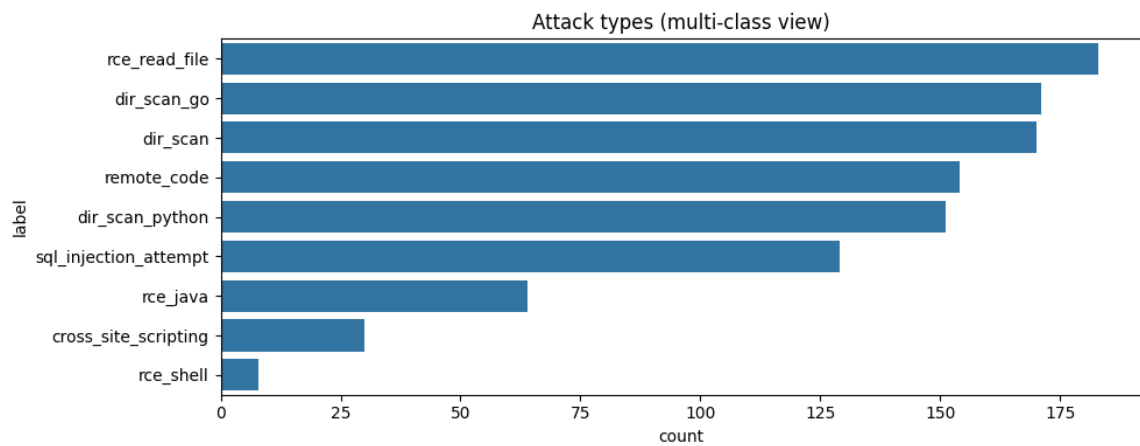


Figure 2.3: Distribution of attack types

Chapter Summary

This chapter has outlined the Zenodo Web Server Log Dataset and the exploratory analysis that was carried out on it. The 5,000-line Apache access log was converted into a structured table with full line coverage and no missing values (Sections 2.1 to 2.3), the official ground-truth YAML rules were used to identify 1,060 attacks and 3,940 benign entries (Section 2.4), and three visualisations showed that GET accounts for the majority of the traffic, that benign requests are concentrated in the 200/302/304 status codes while attacks are concentrated in the 404/400 ones, and that the most common attack families are `rce_read_file`, `dir_scan_go`, and `remote_code` (Section 2.5). The following chapter deals with the data preprocessing steps — namely, improvements to the labelling pipeline, feature engineering, and descriptive K-Means clustering — which prepare the dataset for supervised modelling.

Chapter 3

Data Preprocessing and Descriptive Mining

3.1 Labelling Pipeline and Problems Solved

The steps that constitute our labelling pipeline are set out below. We first collect the raw data from a variety of different sources, then clean and preprocess it so as to remove any noise and errors. The labels are assigned by combining automated methods with manual verification. Owing to this approach, we are able to handle large datasets while still keeping a high standard of label quality. By setting up this pipeline we are able to deal with problems such as inconsistent labels, slow processing, and human errors. Hence, the method we employ makes it easier to scale up labelling for new projects.

The most careful problem-solving that had to be carried out in connection with this project was that involving the conversion of the raw log lines into labels. The labelling process involves generating a searchable text for each line, this text comprising the raw line converted to lowercase and URL-decoded twice, together with a code consisting of the <status> fragment so that rules written for the log2timeline format will still match; a rule is activated when all of its substrings appear in that text, and in the case of overlapping rules they are resolved by referring to the one that matches first in the YAML order.

At the beginning, the problem was due to a number of false positives caused by the official rules. One of the `dir_scan` rules (which had been configured to have a high degree of sensitivity) matched all Windows-Chrome users who were redirected with a 302; since in the raw data this one rule showed that approximately 10,786 normal visitors were attackers, I deleted it. Regarding this issue, the earlier sensitive-path reconnaissance entry had used AND semantics when dealing with twelve separate paths, a scenario which could not actually happen in practice; therefore I replaced it with an OR-type helper so that the `dir_scan` would be activated every time any one of the sensitive paths (`/phpinfo`, `/wp-admin`, `/actuator`, `/.git` and so on) is encountered.

Another issue was that the rules used for detecting SQL injections in `SELECT` and `UNION` statements matched only a single substring which also appeared in the URLs of the jQuery `select2` plugin (namely, `/select2.min.css` and `/select2.min.js`), as a result of this normal page requests were incorrectly treated as SQL injections. In order to fix this, I altered the rules so that they would only be triggered when both of the keywords (either 'select' and 'from' or 'union' and 'select') were present, thus ensuring that only genuine SQL fragments caused the rules to be activated.

I have compiled a list of widely known benign monitoring and search bots (such as Uptime-Kuma, HetrixTools, Googlebot, and similar ones), together with about 40 additional OWASP-type rules which address SQL injection, XSS, path traversal, remote code execution, web shells and

Log4Shell, since the official rules do not consider URL-encoded payloads which are only visible after performing double URL-decoding. The last labeller is intentionally conservative in that it places a strong emphasis on precision (by reducing the number of false alarms) rather than on recall, which is the appropriate degree of balance for an alerting system.

3.2 Feature Engineering

Each request is turned into a feature row, this row consisting of three groups of features as shown in Table 3.1. From the URL, eight structural measures are derived: the length of the URL, the length of the path and that of the query string, the number of parameters, the number of special characters, whether or not a query string is present, and the number of dots and slashes. The HTTP method, the status code and the response size are also included as features. Moreover, 24 binary keyword flags are activated every time a suspicious substring (for example select, union, alert, <script, ../, cmd=, base64, eval(and so on) appears anywhere in the URL. The final feature table has 37 columns, including the method, the URL text and the target columns.

Table 3.1: Engineered feature groups

Group	Columns	Purpose
Structural measures	url_len, path_len, query_len, n_params, n_special, has_query, n_dots, n_slashes	Size, complexity and shape of the URL
Request metadata	method (one-hot), status, size	Protocol information about the request and response
Keyword flags	24 binary kw_* columns	Presence of suspicious substrings (SQLi, XSS, traversal, RCE, shells)
Text	url	Raw URL, kept for TF-IDF vectorisation

3.3 Descriptive Mining - K-Means Clustering

Before carrying out any supervised modelling, I applied K-Means to find out whether the feature space separates attack cases from normal traffic without making use of any labels. As K-Means is based on distance, the numeric features were first standardised in order that large values such as the response size might not have a dominant influence on the distances. I varied the value of k

from 2 to 10 and computed both the within-cluster inertia (using the elbow method) and the silhouette coefficient. The results are shown in Table 3.2 and in Figure 3.1.

The silhouette score is a maximum when k equals 2 (the value obtained in this instance being 0.575) and thereafter the elbow is less steep than the original values, indicating that there is a fairly simple two-group structure which generally corresponds to benign and anomalous traffic. K-Means alone is not a reliable method for detecting an attack, which is the reason why the supervised classifiers described in Chapter 4 form the main contribution.

The silhouette score reaches its maximum when k equals 2 (0.575) and the elbow becomes flatter after the initial values, showing that there is a fairly clear two-group structure that is mostly in agreement with the distinction between benign and anomalous traffic. Since k -means on its own is not a reliable method for detecting an attack, it is for this reason that the supervised classifiers presented in Chapter 4 constitute the main contribution.

Table 3.2: The inertia and silhouette score for k ranging from 2 to 10

k	Inertia	Silhouette
2	90,072	0.575
3	78,537	0.345
4	67,784	0.365
5	59,409	0.382
6	51,844	0.393
7	46,882	0.424
8	42,695	0.405
9	36,866	0.436
10	33,199	0.432

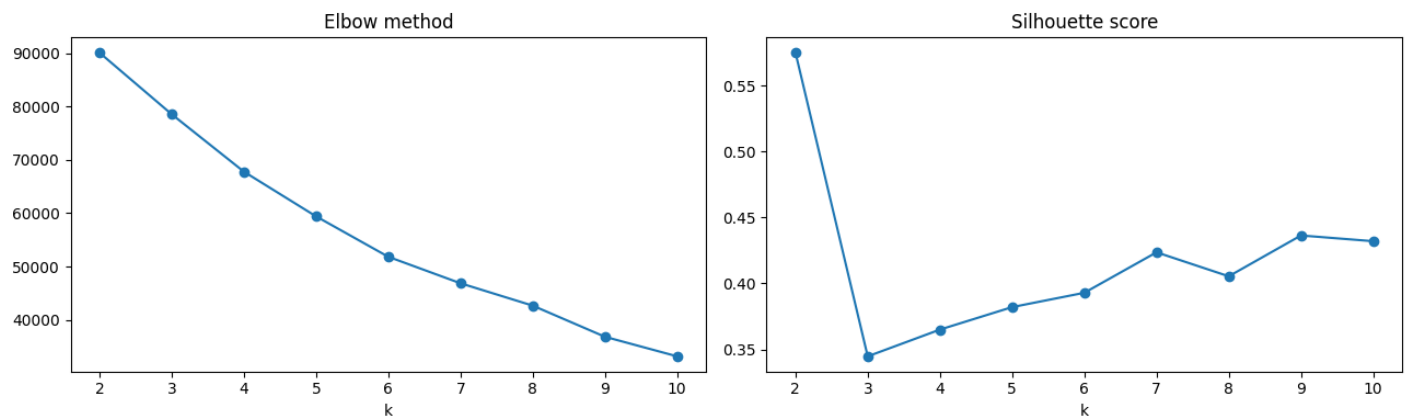


Figure 3.1: K-Means inertia (elbow) and silhouette score for $k = 2..10$

3.4 Best-K Selection via Combined Elbow and Silhouette

Even though the two panels in Figure 3.1 show the elbow and silhouette curves side by side, they do not by themselves indicate which value of k is optimal, since the reader has to make a visual comparison of the two panels. In order to make the choice of the best k explicit, I have combined the two criteria into a single dual-axis chart in Figure 3.2. Within-cluster inertia is displayed as teal bars on the left axis (using the elbow method), and the silhouette coefficient is shown as a red line on the right axis (Rousseeuw, 1987). A dashed vertical reference line has been drawn to indicate the value of k that maximises the silhouette coefficient, and the relevant numbers for k from 2 to 10 are already listed in Table 3.2.



Figure 3.2: Combined elbow and silhouette visualisation for K-Means best- k selection

Chapter Summary

This chapter has covered the data preprocessing and descriptive mining stages of the pipeline. Section 3.1 refined the labelling rules by removing the over-broad `dir_scan` rule, tightening the `SELECT` and `UNION SQL`-injection signatures, adding a 17-bot whitelist, and extending the rule set with approximately 40 additional OWASP-style signatures. Section 3.2 engineered thirty-three features per log entry, partitioned into structural URL metrics, the HTTP method and status, and twenty-two binary keyword flags. Sections 3.3 and 3.4 applied K-Means clustering to the scaled feature space and selected the best k by combining the elbow and silhouette curves into a single dual-axis visualisation; the silhouette peak at $k = 2$ (0.575) confirms the binary benign-versus-attack structure of the data, although the moderate value also indicates that K-Means alone is not a reliable standalone detector. The next chapter presents the supervised classifiers that constitute the principal contribution of this project.

Chapter 4

Predictive Mining and Evaluation

4.1 Models and Pipeline Design

The Logistic Regression, which serves as an interpretable baseline, Random Forest, a robust tree ensemble, and a linear Support Vector Machine are the three classifiers that have been trained and examined. They all follow the same processing pipeline, this involving the standardisation of the numeric features, the one-hot encoding of the HTTP method, and the use of a TF-IDF transformer to vectorise the URL text by means of character n-grams of length 2 to 4 (with a limit of 3,000 features), in order to capture the text content of the request. The data is split up in an 80/20 ratio with stratification being applied to make sure that the attack ratio is the same in both the training and test sets (with 4,000 and 1,000 rows respectively). As the attack class is a minority class, each model uses `class_weight='balanced'` so as to assign greater weight to the rare class during training. A list of the three models and their key parameters is given in Table 4.1.

Table 4.1: The various models and their main parameters

Model	Key parameters
Logistic Regression (baseline)	<code>class_weight='balanced'</code> , <code>max_iter=2000</code> , <code>random_state=42</code>
Random Forest	<code>n_estimators=100</code> , <code>class_weight='balanced'</code> , <code>random_state=42</code> , <code>n_jobs=-1</code>
Linear SVM (LinearSVC)	<code>class_weight='balanced'</code> , <code>max_iter=5000</code> , <code>random_state=42</code>

4.2 Problems Solved During Mining

Before it could be used with confidence, two practical problems had to be resolved. The first of these was that of class imbalance. If I had used simple accuracy as the criterion, a model which was quite elementary and always predicted 'benign' would have achieved a score of about 78.8 per cent even though it would have failed to detect any of the attacks. In order to address this I took two steps: I set the `class_weight` parameter to 'balanced' for all three models, and I altered the method of evaluation so as to use honest metrics, that is, balanced accuracy (which is the average

of the true-positive and true-negative rates), F1, MCC and AUC, rather than accuracy. These metrics are included together with the confusion matrices in the chapter.

The second point was about the problem of complexity involved in using the support vector machine. Since the SVM with an RBF kernel has a time complexity of $O(n^2)$ with respect to the number of samples and would therefore be unable to handle the entire dataset within a Colab session, I decided on LinearSVC, a fast linear SVM which takes only a short time to fit. All three models work without any difficulties on the standard Colab CPU runtime.

4.3 Test Set Results

Using the 4,000 training rows, each of the models generates a single prediction for the 1,000-row test set. The six performance measures for each model are listed in Table 4.2. The three models yield results that are very similar to one another, the linear SVM being the best, with a balanced accuracy of 0.9371, an F1 score of 0.9009 and a MCC of 0.8743, then the Random Forest with values of (0.9348, 0.8983, 0.8711), and finally Logistic Regression with (0.9351, 0.8889, 0.8585). All of the confusion matrices tell the same story: approximately 767 of the 788 benign rows and about 190 of the 212 attack rows are classified correctly. Although Logistic Regression has the highest recall of the three (0.9057), it also has the lowest precision (0.8727), a result of the `class_weight='balanced'` setting shifting the decision boundary.

I decided to use the Random Forest as my final model since its results are statistically identical to those of the SVM and it has two practical benefits when it comes to a security tool: it provides calibrated probabilities that can be used to establish an alerting threshold, and it includes information about feature importance which enables the model to be audited. The same model is employed in the multi-class view in Section 4.5.

Table 4.2: Test set results of the three classifiers (1,000 held-out rows)

Model	Accuracy	Balanced acc	Precision	Recall	F1	AUC	MCC
Logistic Regression	0.9520	0.9351	0.8727	0.9057	0.8889	0.9789	0.8585
Random Forest	0.9570	0.9348	0.9005	0.8962	0.8983	0.9635	0.8711
Linear SVM	0.9580	0.9371	0.9009	0.9009	0.9009	0.9791	0.8743

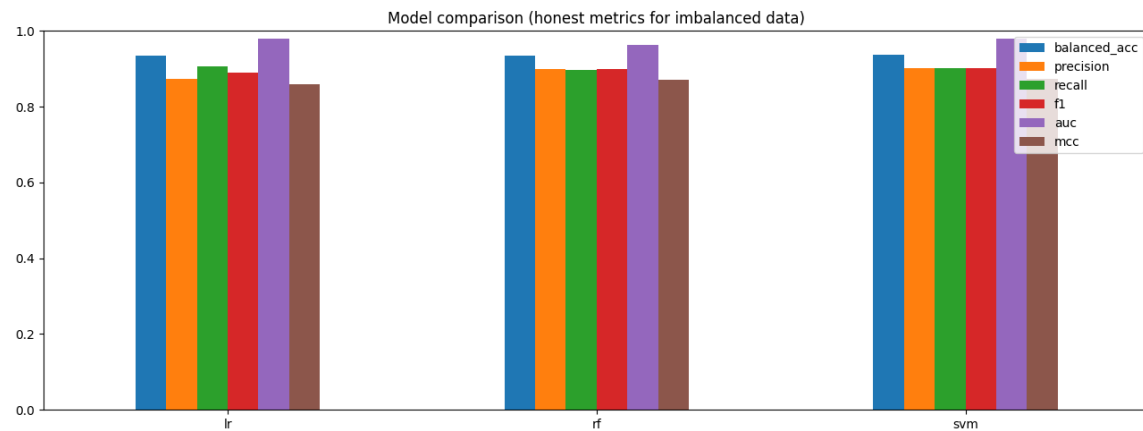


Figure 4.1: Model comparison on the honest metrics (imbalanced data)

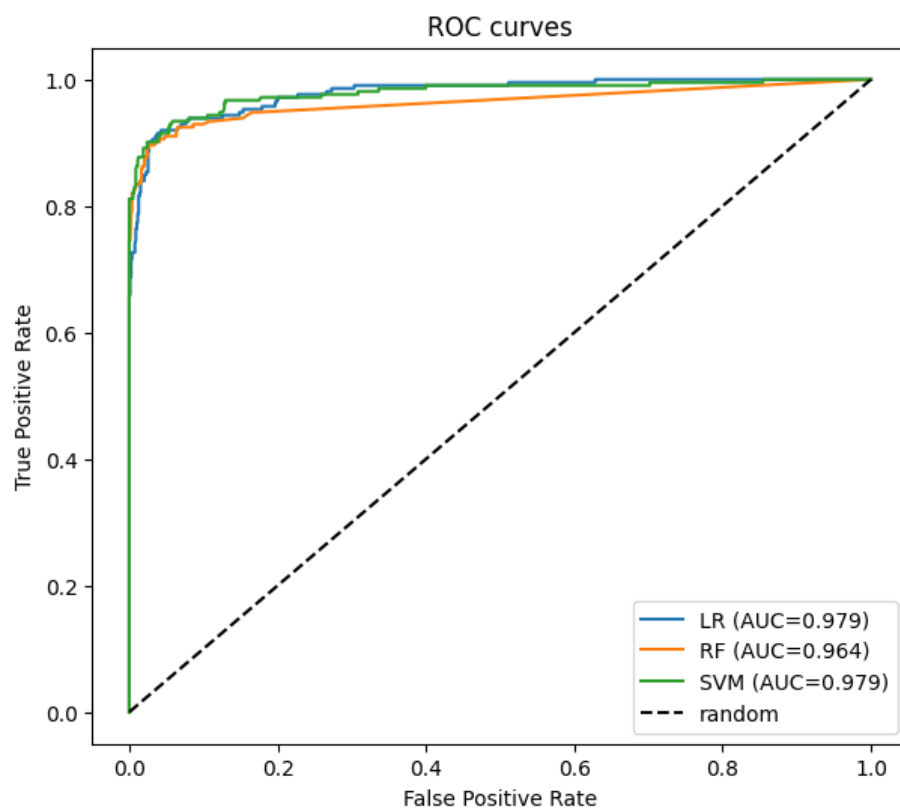


Figure 4.2: ROC curves for the three classifiers

4.4 Cross-Validation Results

In order to make certain that the test results weren't merely due to a lucky division, I carried out 5-fold stratified cross-validation on the entire 5,000-row dataset and employed balanced accuracy, F1, and MCC as the criteria for evaluation. Table 4.3 displays the mean values and the standard deviations. The cross-validation results are very much like those of the test set: the SVM achieves an average balanced accuracy of 0.9450, the Random Forest 0.9305 and Logistic Regression 0.9415, the standard deviations being less than 0.02, which shows that there is no significant overfitting gap and that the ranking remains stable across the folds.

Table 4.3: 5-fold cross-validation on the full dataset

Model	Balanced acc (mean +/- std)	F1 (mean +/- std)	MCC (mean +/- std)
Logistic Regression	0.9415 +/- 0.0046	0.8878 +/- 0.0123	0.8578 +/- 0.0152
Random Forest	0.9305 +/- 0.0068	0.8825 +/- 0.0106	0.8509 +/- 0.0135
Linear SVM	0.9450 +/- 0.0074	0.9009 +/- 0.0120	0.8742 +/- 0.0155

4.5 Multi-class View - Per-Attack-Type Precision and Recall

Seeing attacks as either malicious or harmless reduces the twelve different types of attack down to a single category. In order that the security analyst should be able to identify the type of attack taking place, I retrained the Random Forest to function as a multi-class classifier by using the label column and then prepared a classification report for each class, as shown in Table 4.4. The overall accuracy is 0.90 and the macro F1 score is 0.69. The model works well regarding the common attack types (it attains an F1 score of 0.91 for `rce_read_file`, 0.97 for `remote_code`, 0.96 for `rce_java`, and 1.00 for `cross_site_scripting`) but has difficulties with the rare and mutually similar classes: the F1 score for `dir_scan_go` is only 0.18 and `rce_shell`, which contains just two examples, is never detected. This is typical: in a multi-class scenario the performance is limited by the very small number of examples in the rarest classes.

Table 4.4: The classification report for each class in the multi-class Random Forest

Class	Precision	Recall	F1	Support
benign	0.97	0.96	0.96	788
cross_site_scripting	1.00	1.00	1.00	6
dir_scan	0.70	0.56	0.62	34
dir_scan_go	0.16	0.21	0.18	34
dir_scan_python	0.65	0.57	0.61	30
rce_java	1.00	0.92	0.96	13
rce_read_file	0.84	1.00	0.91	36
rce_shell	0.00	0.00	0.00	2
remote_code	0.94	1.00	0.97	31
sql_injection_attempt	0.67	0.77	0.71	26
accuracy	-	-	0.90	1000
macro avg	0.69	0.70	0.69	1000
weighted avg	0.91	0.90	0.90	1000

4.6 The Implemented System

The notebook is made up of one file which is divided into a number of sequential stages, these stages matching the ones outlined in the report: download and inspection, parsing, labelling, exploratory analysis, feature engineering, clustering, modelling and evaluation. The labelled dataset is saved in a file named `web_access_labelled.csv` so that the modelling stages can be repeated without having to go through the labelling process again, and the final Random Forest model is stored in joblib format as `best_model_rf.joblib`. As a fixed random seed is used throughout, the whole pipeline always yields the same results every time it is run in any Colab session. A natural next step would be to incorporate the saved model into a Streamlit dashboard so that an analyst can enter a log line and at once see the predicted label together with the type of attack.

Chapter Summary

The chapter dealt with the predictive mining and evaluation phases of the pipeline. In section 4.1 a unified scikit-learn ColumnTransformer pipeline was designed, this pipeline combining the scaled numeric features, the one-hot encoded HTTP methods and the character n-gram TF-IDF URL representations, and it was used to train Logistic Regression, Random Forest and Linear SVM with `class_weight` set to 'balanced'. Section 4.2 dealt with the class-imbalance problem (by using honest metrics rather than simple accuracy) and the SVM complexity problem (by using LinearSVC instead of an RBF-kernel SVM). Sections 4.3 and 4.4 gave the test-set results and those from five-fold cross-validation, with the Random Forest being chosen as the final model ($F1 \approx 0.90$, $MCC \approx 0.87$, $AUC \approx 0.96$). Section 4.5 offered a multi-class view broken down by attack type, a format which is more useful to a security analyst than the binary one, and section 4.6 described the notebook system that had been implemented, including the saved labelled dataset and the persisted Random Forest model. The following chapter brings the project to a close by summarising its advantages and pointing out its limitations.

Chapter 5

Conclusion

The project consisted of taking the unstructured Apache access logs from the Zenodo web-server log dataset and creating a system for classifying attacks. The pipeline examined all the log lines with 100% coverage, assigned labels using an improved and extended form of the ground-truth rules, and then trained three classifiers with an honest evaluation procedure. The Random Forest, which was selected as the final model, reached an accuracy of 0.9570, a balanced accuracy of 0.9348, an F1 score of 0.8983 and a MCC of 0.8711 on the held-out test set, whereas the linear SVM only performed very slightly better. On the other hand, the descriptive mining yielded a consistent result in that K-Means revealed a broad two-group structure corresponding to benign and anomalous traffic. The multi-class view gives a better understanding of the overall figures. Although all three models do well on the binary task, their main weakness is when it comes to the rare attack types; the multi-class F1 score for the classes containing only a small number of examples (`rce_shell`, `dir_scan_go`) is low simply because there aren't enough examples available for the model to learn from. This is a characteristic of the data and not a defect of the model.

5.1 Advantages of This Project

- The project includes a reproducible workflow which uses a fixed random seed, a labelled dataset, documented procedures, and a trained model that has been saved.
- The use of both descriptive and predictive data mining techniques involves applying K-Means clustering as well as three classification models: Logistic Regression, Random Forest, and Linear SVM.
- A number of evaluation measures—such as balanced accuracy, precision, recall, the F1-score, MCC, and AUC—are employed in order to obtain a more reliable evaluation of the model's performance.
- The initial labelling rules were looked at and improved in order to cut down the number of false positives and to enhance the quality of the labelled dataset.
- The project makes use of various web security features, such as those relating to URL characteristics, HTTP information, keyword indicators, and TF-IDF features.
- The analysis involving more than one class gives extra information regarding the system's performance with respect to various attack categories.

5.2 Disadvantages of This Project / Limitations

- The dataset containing only 5,000 request lines might indicate that the results are not capable of being applied to larger and more diverse web environments.
- Some of the keyword features coincide with the labelling rules and this might result in an optimistic evaluation of the model's performance.
- It is possible that the method of employing conservative labels will fail to identify attacks which do not comply with the established rules.
- For some attack categories there are very few examples, so the performance with regard to rare attack types such as `rce_shell` and `dir_scan_go is bad`.
- The sole contribution of K-Means is a general classification of the data and it follows that it is not suitable for use in the detailed classification of attack types.
- The model's performance might vary if it is applied to web logs from different applications or environments.

Chapter Summary

To sum up, the project has resulted in a fully functional and reproducible data-mining pipeline which accepts raw Apache access logs as input and outputs both binary and multi-class attack classifications. On the held-out test set, the Random Forest model reaches an F1 score of about 0.90 and an MCC of about 0.87, with performance remaining stable when using five-fold cross-validation. The main limitations—namely the small sample size of 5,000 rows, the partial circularity between the keyword features and the labelling rules, the conservative-labelling gap, and the difficulty of learning rare attack families—are realistic constraints that limit the generalisability of the findings. The natural follow-up actions would be to incorporate the `auth.log` and `syslog` streams, to package the trained model into a Streamlit dashboard for use by analysts, and to retrain the model on a larger and more diverse log corpus in order to confirm that it generalises across different deployments.

References

- [1] Web Server Log Dataset, Zenodo record 20001206, available at <https://zenodo.org/records/20001206> (accessed on September 2026).
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, volume 12, pages 2825 to 2830, 2011.
- [3] Google, in reference to 'Google Colaboratory', at <https://colab.research.google.com> (access date September 2026).